1. Project Title

Adaptive LASSO using Proximal Gradient Descent with Dynamic Soft Thresholding

This project builds a custom regression model that predicts house prices while also selecting which input features are really important. It does this using an optimization method called proximal gradient descent combined with a dynamic (changing over time) version of LASSO regularization.

---

2. Project Goal

The main problem we are solving is:

> **Predict house prices accurately while reducing the number of unnecessary features.**

In the House Prices dataset there are many features: room counts, quality scores, neighborhood labels, etc. After preprocessing and one-hot encoding, we have around **267 input features**. Not all of them are truly useful.

Why feature selection is important

- **Improves interpretability**
  If only a small number of features have non-zero coefficients, it is easier to explain which factors affect the price.

- **Reduces overfitting**
  Using too many weak or noisy features can cause the model to memorize the training data

and perform worse on unseen data. Selecting a smaller set of strong features usually generalizes better.

- **Simplifies models**
  A sparser model (with many zero coefficients) is cheaper to store, faster to evaluate, and easier to debug or extend.

This project explores how **numerical optimization techniques** (in particular proximal gradient descent with dynamic regularization) can improve feature selection compared to standard methods.

---

3. Dataset Used

The dataset used in this project is:

House Prices: Advanced Regression Techniques (Popular Kaggle dataset)

Each row in the dataset corresponds to a house sale. The features include:

- **House characteristics**
  Floor area, number of rooms, number of bathrooms, presence of a garage or basement, etc.

- **Neighborhood information**
  Which neighborhood the house is in, zoning, surrounding conditions.

- **Building quality and condition**
  Overall quality scores, materials, finishing, and maintenance condition.

- **Lot and exterior details**
  Lot size, lot shape, alley access, fence, pool quality (when available), and other exterior features.

The target variable is the **sale price** of the house (log-transformed in our preprocessing).

Preprocessing steps

All preprocessing logic lives in `src/data_preprocessing.py`. The main steps are:

- **Removing outliers**
  Extreme outliers in living area (`GrLivArea` > 4000) are removed to avoid distorting the regression.

- **Handling missing values**

  - Numerical columns: missing values are filled with the median of each column.
  - Categorical columns: missing values are filled with the most frequent category.

- **Encoding categorical features**
  Categorical columns (like neighborhood) are converted into numerical vectors using **one-hot encoding**. Each category becomes its own binary feature. This is why the final feature count grows to about **267 features**.

- **Scaling numerical features**
  Numerical columns are standardized so they have roughly zero mean and unit variance. This helps gradient-based optimization converge more smoothly.

- **Splitting train/test data**
  The dataset is split into training and test sets so we can evaluate generalization performance using **test Mean Squared Error (MSE)**.

After all these steps, we obtain:

- `X_train` and `X_test`: preprocessed design matrices with about 267 features.
- `y_train` and `y_test`: log-transformed house prices.

---

4. Models Compared in the Project

We compare four regression models:

1. **Linear Regression**

   - Basic least-squares model with **no regularization**.
   - Tries to fit the best straight-line relationship in this high-dimensional feature space.
   - Uses all 267 features (no feature selection).

2. **Ridge Regression**

   - Linear regression with **L2 regularization** (penalizes large weights but does not set them exactly to zero).
   - Helps reduce overfitting but typically keeps all features non-zero.

3. **Standard LASSO**

   - Linear regression with **L1 regularization**.
   - The L1 penalty encourages some weights to become exactly zero → performs **feature selection**.
   - Uses a fixed regularization strength (a single lambda value for the whole training).

4. **Adaptive LASSO (our implementation)**

   - Uses the **same L1 penalty idea**, but the regularization strength is **dynamic** and changes over time during training.
   - Implemented using **proximal gradient descent** plus a **dynamic soft-thresholding** rule.
   - Aims to get a good balance between sparsity (few features) and prediction accuracy.

All four models are trained on the **same preprocessed dataset**, so their results are directly comparable.

---

5. Main Idea of the Project

Limitation of standard LASSO

Standard LASSO uses a **fixed regularization parameter** λ throughout training:

- If λ is **large**, the model becomes very sparse (many zero coefficients), but it may also remove important features and hurt accuracy.
- If λ is **small**, the model keeps more features and may achieve lower error, but sparsity and interpretability are worse.

There is no way to "prune strongly at the beginning and relax later" with a single fixed λ.

Improvement introduced in this project

Our Adaptive LASSO uses a **dynamic regularization schedule**:

$$\lambda_t = \frac{\lambda_0}{\sqrt{t}}$$

Here:

- λ_0 is the initial (strong) regularization strength.
- t is the iteration number during training.

Interpretation:

- **Early iterations** (small t) → λ_t is large → **strong shrinking of coefficients**. Many unimportant features are pushed quickly toward zero.
- **Later iterations** (large t) → λ_t becomes smaller → regularization gets weaker, and the model focuses more on **fine-tuning** the remaining non-zero coefficients.

This dynamic behaviour gives a different sparsity pattern compared to standard LASSO with a fixed λ.

---

6. Optimization Algorithm

The Adaptive LASSO optimizer is implemented in `src/adaptive_lasso.py` using **proximal gradient descent**. In simple terms, each training iteration does two steps:

Step 1: Gradient descent to reduce prediction error

- The model first updates its weights to reduce the **prediction error** (difference between predicted price and true price).
- This is a standard gradient descent step, moving the weights in the direction that reduces the mean squared error on the training data.

Step 2: Soft-thresholding to shrink coefficients and remove features

- After the gradient step, the model applies a **soft-thresholding** operation to the weight vector.
- Conceptually:
  - If a weight is **small in magnitude**, it is set to **exactly zero**.
  - If a weight is **large**, it is shrunk slightly toward zero but stays non-zero.

This step is what performs **feature selection**:

- Weights that become exactly zero correspond to **features that are effectively removed** from the model.
- Over many iterations, the algorithm gradually builds a sparse set of important features.

Because the threshold used in soft-thresholding is based on the **dynamic** λ_t, the aggressiveness of feature removal changes over time (stronger early, milder later).

---

7. Code Structure

The project code is organized as follows.

`src/data_preprocessing.py`

- Loads the House Prices dataset from OpenML.
- Removes extreme outliers and drops some mostly empty columns.
- Handles missing values (numerical and categorical).
- Encodes categorical features using **one-hot encoding**.
- Scales numerical features using **standardization**.
- Splits the data into training and test sets.
- Returns the processed feature matrices and target arrays.

`src/baselines.py`

- Runs the three baseline regression models:
  - Linear Regression,
  - Ridge Regression,
  - Standard LASSO.
- Computes:
  - Test **Mean Squared Error (MSE)**,
  - **Number of non-zero coefficients** for each model.
- Provides both a reusable function (`compute_baseline_metrics`) and a script entry point (`run_baselines`).

`src/adaptive_lasso.py`

- Contains the main **Adaptive LASSO optimizer** implementation as a Python class `AdaptiveLasso` .
- Key responsibilities:
  - Maintain model parameters (`coef_` and `intercept_`).
  - Implement the **training loop** using proximal gradient descent.
  - Apply **dynamic lambda scheduling** ($\lambda_t = \lambda_0/\sqrt{t}$, etc.).
  - Apply the **soft-thresholding** operator to enforce sparsity.
  - Record training diagnostics such as:
    - Loss values over iterations,
    - Number of non-zero features over iterations,
    - Lambda values over iterations.

`src/train_adaptive_lasso.py`

- Runs the **full experiment pipeline** end-to-end:
  - Loads and preprocesses the dataset via `get_processed_data` .
  - Trains the `AdaptiveLasso` model with chosen hyperparameters.
  - Evaluates Adaptive LASSO on the test set (MSE and sparsity).
  - Trains and evaluates the baseline models using `compute_baseline_metrics` .
  - Aggregates all metrics into a single dictionary.
  - Saves metrics and plots to the `results/` folder.
  - Prints a results table to the console.

This is the main script you run to reproduce the experiment.

`src/visualization.py`

- Generates graphs to help analyze model behaviour:
  - **Loss vs iterations**:
    - Shows how quickly and smoothly the optimization converges.
  - **Sparsity vs iterations**:
    - Shows how the number of active (non-zero) features changes during training.
  - **Model comparison chart**:
    - Compares MSE and number of non-zero features across all models.

`results/` folder

- After running the main training script, this folder contains:
  - `metrics.json`
    - Stores the final MSE and non-zero feature counts for each model, plus the Adaptive LASSO hyperparameters.
  - `loss_curve.png`
    - Plot of loss (error + regularization) versus training iterations for Adaptive LASSO.
  - `sparsity_curve.png`
    - Plot of the number of non-zero features versus training iterations.
  - `model_comparison.png`
    - Bar chart showing test MSE and number of non-zero features for all four models.

---

8. Experimental Results

Metrics measured

We focus on two main metrics:

- **Mean Squared Error (MSE)**
  Measures how far the predicted log house prices are from the true values on average. Lower is better.

- **Number of non-zero features**
  Counts how many coefficients in the model are non-zero. This tells us how **sparse** the model is.

Final results (from the latest run)

- **Linear Regression**
  - MSE ≈ **0.0167**
  - Features used: **267** (all features)
- **Ridge Regression**
  - MSE ≈ **0.0150**
  - Features used: **267** (all features, just shrunk)
- **Standard LASSO**
  - MSE ≈ **0.0200**
  - Features used: **16**
  - Very sparse model with strong feature selection.
- **Adaptive LASSO (ours)**
  - MSE ≈ **0.1039**

- Features used: **91**
- Moderately sparse model with dynamic regularization.

Interpreting the results in simple terms

- **Ridge** has the **lowest MSE**, but it keeps **all 267 features**, so it does not help interpretability.
- **Standard LASSO** uses only **16 features**, which is extremely sparse and very interpretable, with a small increase in MSE compared to Ridge.
- **Adaptive LASSO** uses **91 features**, so it is less sparse than Standard LASSO but still much sparser than Linear or Ridge. Its MSE is higher than the others, showing that our chosen hyperparameters favour sparsity more strongly than accuracy.

These results illustrate the classic **trade-off between accuracy and sparsity**:

- If you want **maximum accuracy**, Ridge (or even Linear Regression) is best.
- If you want **maximum sparsity**, Standard LASSO with strong regularization is best.
- Adaptive LASSO provides a **different sparsity pattern** based on its dynamic schedule. By tuning its hyperparameters, we can move it closer to either side of this trade-off.

---

9. Key Findings

From this project, we can summarize the following key points:

- **Adaptive LASSO gradually reduces features during training**
  Because the regularization strength starts high and decays over iterations, many features are pruned early, and only a subset remains active later.

- **It produces a model with fewer features than unregularized regression**
  Compared to Linear and Ridge Regression (267 features), Adaptive LASSO ends up using only 91 features, which is a significant reduction.

- **Dynamic regularization behaves differently from standard LASSO**
  Standard LASSO uses one fixed penalty and tends to “decide” the sparsity pattern more uniformly. Adaptive LASSO changes its penalty over time, which leads to a different path of feature selection and potentially offers more control over when and how features are pruned.

Overall, the project shows that **changing the regularization strength during optimization** is a viable technique for exploring different sparsity behaviours and understanding the trade-off between model simplicity and accuracy.

---

10. How to Run the Project

You can reproduce the experiments with just a few commands.

Step 1: Install dependencies

Make sure you have Python 3.9+ installed, then run:

```
pip install numpy pandas scikit-learn matplotlib
```

Step 2: Run the experiment

From the project root directory, run:

```
python src/train_adaptive_lasso.py
```

This script will:

- Download and preprocess the House Prices dataset (if not already cached).

- Train all four models:
  - Linear Regression,
  - Ridge Regression,
  - Standard LASSO,
  - Adaptive LASSO (with dynamic regularization).
- Compute and print the **MSE** and **number of non-zero features** for each model.
- Save the results to the `results/` folder:
  - `metrics.json` (numerical results),
  - `loss_curve.png` ,
  - `sparsity_curve.png` ,
  - `model_comparison.png` .

You can open these files to quickly understand how the models behaved.

---

11. Conclusion

This project demonstrates how **numerical optimization techniques** can be used not only to fit regression models, but also to **control feature selection** in a principled way.

By implementing Adaptive LASSO with **proximal gradient descent** and a **dynamic soft-thresholding schedule**, we:

- Built a complete pipeline from data preprocessing to model training and visualization.
- Compared multiple regression models on a real-world dataset.
- Observed how dynamic regularization affects sparsity and accuracy.